DAY-2 Hands-On Activities

PROBLEM 1: Student Marks Analyzer (LEVEL-1)

Scenario:

You are developing a small utility for a teacher to analyze student marks stored in an array.

📌 Requirements

- Store student marks in an array.
- Calculate total and average marks using array methods.
- Display pass/fail result based on average.
- Use let/const appropriately.
- Use arrow functions for calculations.
- Display output using template literals.

🛠️ Technical Constraints

- Must use array methods like reduce() and map().
- Use only ES6+ syntax.
- No external libraries.
- All logic must be inside modular JavaScript file.

🎯 Learning Outcome

- Understand array processing.
- Practice ES6 features.
- Learn modular coding structure.

PROBLEM 2: Product Cart Summary (LEVEL-1)

Scenario:

Build a simple shopping cart summary system.

📌 Requirements

- Store product objects (name, price, quantity) in an array.
- Calculate total cart value.
- Display formatted invoice using template literals.
- Use arrow functions.
- Export calculation function from a module.

🛠️ Technical Constraints

- Use map() and reduce().
- Use ES6 modules (export/import).
- No DOM manipulation required.

🎯 Learning Outcome

- Work with arrays of objects.
- Understand modules.
- Improve functional programming skills.

PROBLEM 3: Weather Data Fetcher (LEVEL-2)

Scenario:

Create an application that fetches weather data asynchronously.

📌 Requirements

- Fetch data from a public API using fetch().
- Implement version using Promises.
- Implement version using async/await.
- Handle errors properly.
- Display formatted weather report.

🛠️ Technical Constraints

- Use async/await.
- Use template literals.
- Use arrow functions.
- Must include proper error handling using try/catch.

🎯 Learning Outcome

- Understand Promises and async/await.
- Handle asynchronous operations.
- Improve API handling skills.

PROBLEM 4: Task Manager with Async Storage (LEVEL-2)

Scenario:

Develop a task manager where tasks are saved and retrieved asynchronously.

📌 Requirements

- Store tasks in an array.
- Create addTask(), deleteTask(), and listTasks() functions.
- Simulate async storage using setTimeout with callbacks.
- Convert callback-based logic into Promises.
- Refactor into async/await version.
- Use ES6 modules.

🛠️ Technical Constraints

- Must demonstrate callback → promise → async/await evolution.
- Use let/const properly.
- Use arrow functions.
- Use template literals for display.

🎯 Learning Outcome

- Deep understanding of async JavaScript.
- Learn refactoring async code.
- Master modern ES6+ features.